

B1B13PPS

Průmyslové počítačové systémy

Michal Brejcha

`brejcmic@fel.cvut.cz`

ČVUT v Praze, FEL

Praha, 2025

Obsah

- 1 Funkce - podprogram
- 2 Rekurzivní funkce
- 3 Přerušení
- 4 Program TIM2 s přerušením

Téma

- 1 Funkce - podprogram
- 2 Rekurzivní funkce
- 3 Přerušení
- 4 Program TIM2 s přerušením

Funkční volání

- Část podprogramu je volána opakovaně z různých míst programu:
 - ⇒ musí se chovat stejně jako skoky,
- po provedení podprogramu pokračuje hlavní program z místa, kde byla funkce volána:
 - ⇒ je nutné uchovávat návratovou hodnotu,
- může obsahovat parametry:
 - ⇒ předání pomocí určených registrů (A, X, ...),
 - ⇒ předání pomocí zásobníku (STACK),
- může mít návratovou hodnotu:
 - ⇒ téměř výhradně pomocí určeného registru (A, X, ...).

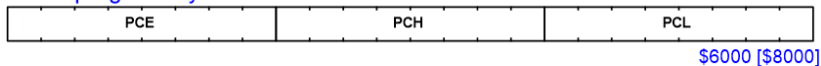
Instrukce CALL

Příklad: **CALL** tato_funkce ; skok na řádek tato_funkce

- Instrukce reprezentuje absolutní skok,
- **Program Counter** (programový čítač) je nejdříve zvýšen o délku instrukce **CALL** (viz dále),
- do zásobníku se uloží v pořadí hodnota **PCH** a **PCL** (spodní a prostřední byte **Program Counter**). Obdoba volání:

- 1 **PUSH** PCL,
- 2 **PUSH** PCH,

PC ... programový čítač



- aktuální stav **PCH:PCL** je přepsán příslušnou adresou „tato_funkce“,

Délka a trvání instrukce CALL - programovací manuál

CALL

CALL Subroutine (Absolute)

CALL

Description The current PC register value is pushed onto the stack, then PC is loaded with the destination address in same section of memory. The CALL destination and the instruction following the CALL should be in the same section as PCE is not stacked. The corresponding RET instruction should be executed in the same section. This instruction should be used versus CALLR when developing a program.

dst	Asm	cy	lgth	Op-code(s)				ST7
longmem	CALL \$1000	4	3	CD	MS	LS		x
(X)	CALL(X)	4	1	FD				x
(shortoff,X)	CALL(\$10,X)	4	2	ED	XX			x
(longoff,X)	CALL(\$1000,X)	4	3	DD	MS	LS		x
(Y)	CALL(Y)	4	2	90	FD			x
(shortoff,Y)	CALL(\$10,Y)	4	3	90	ED	XX		x
(longoff,Y)	CALL(\$1000,Y)	4	4	90	DD	MS	LS	x
[shortptr.w]	CALL[\$10.w]	6	3	92	CD	XX		x
[longptr.w]	CALL[\$1000.w]	6	4	72	CD	MS	LS	
([shortptr.w],X)	CALL([\$10.w],X)	6	3	92	DD	XX		x
([longptr.w],X)	CALL([\$1000.w],X)	6	4	72	DD	MS	LS	
([shortptr.w],Y)	CALL([\$10.w],Y)	6	3	91	DD	XX		x

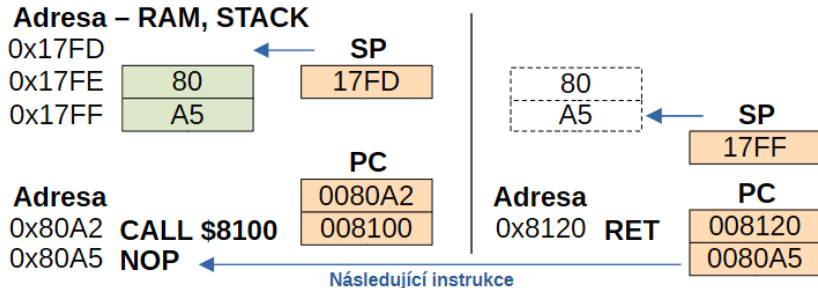
Instrukce RET

Příklad: RET ; skok na adresu načtenou ze zásobníku

- Instrukce reprezentuje absolutní skok - obnovuje původní stav **PCH:PCL**,
- ze zásobníku se získá v pořadí hodnota **PCH** a **PCL** a uloží se do Program Counter. Obdoba volání:
 - 1 **POP** PCH,
 - 2 **POP** PCL,
- **PCE** se nemění, tudíž cíl skoku i návratová adresa musí být ve stejném sekci adresy **PCE**,
- pro adresování celé paměti (delší skoky) lze použít kombinaci instrukcí **CALLF** a **RETF**, které upravují celou hodnotu **PCE:PCH:PCL**,
- varianta s relativním adresováním je s instrukcí **CALLR** - spoří místo v paměti (2 byte),

Funkce a zásobník

Návratová adresa je uložena v zásobníku:



K datům v zásobníku lze přistupovat indexovým adresováním. Toho lze využít několika způsoby:

- pomocí zásobníku můžeme předávat parametry funkce,
- v zásobníku můžeme vytvářet lokální proměnné dané funkce,
- můžeme měnit návratovou adresu funkce.

Jazyk C, překladač pro STM8 COSMIC

Volání funkce:

- Název funkce v **C** je na začátku překladačem doplněn o znak „_“ (krátká adresa - **CALL**) nebo o „f_“ (dlouhá adresa - **CALLF**),
- pokud má funkce jeden parametr, tak je předán pomocí reistru **A** (BYTE) nebo **X** (WORD) a po skoku instrukcí **CALL** je uložen do zásobníku,
- další parametry se ukládají do zásobníku před voláním instrukce **CALL**

Příklad: (char = byte)

char tatoFunkce(**char** arg1, **char** arg2, **char** arg3)

STACK:

SP	SP+1	SP+2	SP+3	SP+4	SP+5
—	arg1	PCH	PCL	arg2	arg3

Jazyk C, překladač pro STM8 COSMIC

Lokální proměnné

- vytvářejí se zásadně v zásobníku, proměnná má platnost jen po dobu trvání funkce,
- funkce může volat sama sebe - **REKURZE**.

Příklad: `char tatoFunkce(void) {`

```

    char    promenna0 = 0;
    char    promenna1 = 0;
    char    promenna1 = 0;
    return  0; }

```

STACK:

SP	SP+1	SP+2	SP+3	SP+4	SP+5
-	promenna2	promenna1	promenna0	PCH	PCL

Jazyk C, překladač pro STM8 **COSMIC**

Návratová hodnota

- u většiny překladačů (podle typu procesoru) se používá některý ze systémových registrů,
- registr **A** je pro návratovou hodnotu velikosti 1 BYTE nebo stejně velký ukazatel,
- registr **X** je pro návratovou hodnotu velikosti 1 WORD nebo stejně velký ukazatel,
- pro více bytové návratové hodnoty je vyhrazeno speciální paměťové místo v RAM (**c_lreg**).

Téma

- 1 Funkce - podprogram
- 2 Rekurzivní funkce**
- 3 Přerušení
- 4 Program TIM2 s přerušením

Rekurze

- Proces při kterém se objekt nebo funkce odkazuje sama na sebe,
- funkce odkazující na sama sebe je **rekurzivní funkcí**,
- sama rekurze je využívána v případech, kdy problém lze dělit na menší celky se stejným algoritmem řešení.

Příklad: Výpočet Fibonacciho posloupnosti

$$F_n = F_{n-1} + F_{n-2}$$

$$F_0 = 0$$

$$F_1 = 1$$

- na procesu výpočtu ukážeme předání parametru, založení lokální proměnné a vrácení hodnoty.

Příklad programu Fibonacci v jazyce C

```

1 void fibona(int n) {                                // funkce fibona
2
3     if(n < 2) {
4         return n;                                    // nepotřebujeme lokální proměnné,
5                                                         // rovnou vrátíme parametr
6     }
7     else {
8         return fibona(n-1) + fibona(n-2);            // zde jsou nutné lokální proměnné
9                                                         // pro uložení mezivýsledků
10    }
11 }
12
13 #define TOP_IDX                                     13    // maximální index výpočtu
14
15 int main() {
16     for (int idx = 0; idx <= TOP_IDX; idx++) // hlavní smyčka
17     {
18         printf("%d ", fibona(idx));                // tisk hodnoty
19     }
20     return 0;
21 }

```

- V assembleru zachováme význam proměnných a konstant,
- výpis **printf** nahradíme zobrazením na LED přípravku.

Program Fibonacci - Definice symbolických konstant

```

10 ;-----
11 |          BYTES
12 ; konstanty pro nastavení pinu portu B = LED
13 INI_PB_DDR    equ    %11111111    ; vse vystup
14 INI_PB_CR1    equ    %11111111    ; vse push/pull
15 INI_PB_ODR    equ    %11111111    ; 1 zhasne LED
16
17 ; cilova pozice ve Fibonacciho posloupnosti
18 TOP_IDX      equ    13
19
20 ; konstanty funkce wait
21 INI_POC1      equ    5            ; pocet pruchodu
22 |          |          |          ; vnejsiho cyklu
23 |          WORDS
24 INI_Y         equ    50000        ; pocet pruchodu
25 |          |          |          ; vnitřního cyklu
26

```

Maximální index posloupnosti

- Definujeme **TOP_IDX**.

Program Fibonacci - Inicializace

```

27 ;      PROMENNE
28 ;-----
29      segment 'ram0'
30
31 ; promenna funkce wait v RAM
32 poc1      ds.b      1                      ; rezervace 1 byte
33
34 ; aktualni poyice ve Fibonacciho posloupnosti
35 idx      ds.b      1
36
37 ;      PROGRAM
38 ;-----
39      segment 'rom'
40
41 ; inicializace vystupu LED
42 main.l    mov      PB_ODR, #INI_PB_ODR      ; stav vystupu pinu
43           mov      PB_CR1, #INI_PB_CR1      ; typ totemu pinu
44           mov      PB_DDR, #INI_PB_ODR      ; smer pinu
45

```

Index, globální proměnná v hl. smyčce

Inicializace HW

Program Fibonacci - Hlavní smyčka

```

46      ; inicializace hodnot promennych
47 start  mov     idx, #0                ; idx=0
48      ld      A, idx                  ; A = idx
49      A je parametrem funkce fibona
50      ; smyčka dokud je idx <= TOP_IDX
51 smyčka call    fibona                ; volání funkce
52      cpl     A                      ; negace A
53      ld      PB_ODR, A              ; vystup LED = A
54      call    wait                   ; čeká 0,5 s
55      inc     idx                    ; idx = idx+1
56
57      ; test konce
58      ld      A, idx                  ; A = idx
59      cp      A, #TOP_IDX             ; test (A-TOP_IDX)
60      jr      smyčka                 ; skok při
61      |      |                       ; (A-TOP_IDX) < 0
62      jp      start                  ; skok na start

```

- Provedeme **TOP_IDX+1** průchodů a výsledky funkce **fibona** zobrazíme na LED.

Program Fibonacci - Funkce fibona

64	fibona	cp	A, #1		; test (A-1)
65		jrugt	vice		; A > 1
66		ret			; vraci A
67	vice	dec	A		; A=A-1
68		push	A	Vzniká lokální proměnná	; (SP) = A, SP+1
69		call	fibona		; fibona(A-1)
70		push	A		; ulozeni vysledku
71		ld	A, (\$02, SP)		; A = idx
72		dec	A		; A=A-1
73		call	fibona		; fibona(A-2)
74		add	A, (\$01, SP)		; secteni vysledku
75		ld	(\$02, SP), A		; ulozeni vysledku
76					; na nejvzdalenejsi
77					; pozici ve STACKu
78					; obnova SP a nahrani navratove hodnoty
79		pop	A		
80		pop	A		; spravny vysledek
81		ret			

Doporučení

- Program krokujte v simulátoru nebo v přípravku,
- zobrazte si paměť **RAM** na pozici **0x07FF** (oblast zásobníku - **STACK**),
- pozorujte jak roste využití zásobníku,
- v extrémním případě vysokého indexu může dojít k vyčerpání celého vyhrazeného prostoru a tím k přetečení zásobníku
⇒ **STACK OVERFLOW**.

The screenshot shows the 8086 assembly editor with the following code in the main window:

```

50      ; smycka dokud je idx <= TOP_IDX
51      smycka  call  fibona      ; volani funkce
52      ← cpl     A               ; negace A
53      ld      PB_ODR, A        ; vystup LED = A
54      call    wait             ; cekat 0,5 s
55      inc     idx              ; idx = idx+1

```

The file name is `fibonacci.asm`. The memory dump on the right shows the following values:

```

0007C0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
0007D0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
0007E0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
0007F0 FF FF FF FF FF FF 80 BC 01 80 BC 01 02 80 96
000800 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
000810 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
000820 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
000830 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF

```

Below the code editor, the status bar shows:

- Program Counter: PC 0x8096
- Stacks: SP 0x07ff
- Index registers: X 0x00, Y 0x00
- Accumulator: A 0x02
- Condition Flags: I Level 0x03, V ☐, IF ☒, H ☐, IOPL ☒, N ☐, Z ☐, C ☐

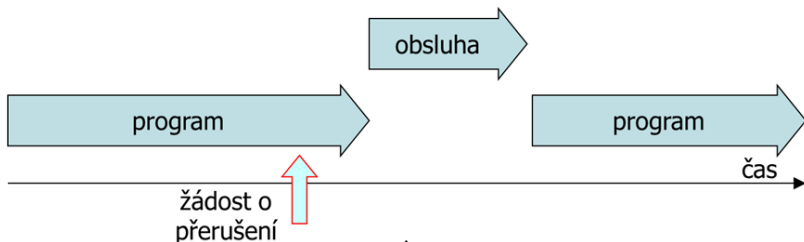
A red text overlay on the right side of the image reads: **Funkce s každým voláním využívá stále více paměti zásobníku, zde pro "idx = 3"**

Téma

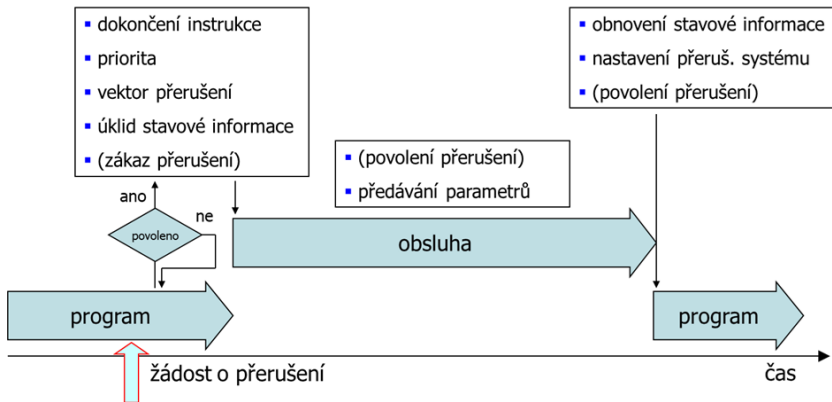
- 1 Funkce - podprogram
- 2 Rekurzivní funkce
- 3 Přerušení**
- 4 Program TIM2 s přerušením

Přerušovací systém procesoru

- zajišťuje reakci na asynchronní (často vnější) událost,
- při události je přerušeno vykonávání stávajícího programu a přejde se k „obsluze“ této události,
- po skončení přerušeni se vrací procesor tam, kde opustil původní program,
- u každého MCU existuje způsob jak globálně všechny zdroje přerušeni povolit i zakázat (globální povolení).



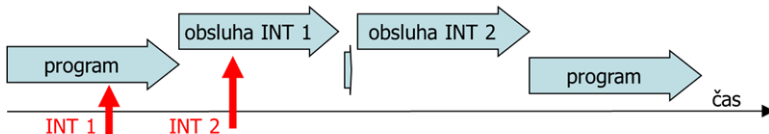
Vznik žádosti o přerušeni



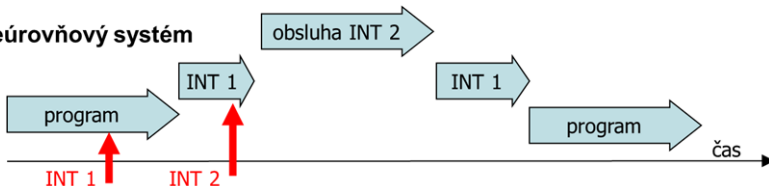
- U většiny zdrojů je nutné přerušeni povolit - **maskovatelné přerušeni**,
- výjimkou je například SW přerušeni TRAP - **nemaskovatelné přerušeni**.

Zpracování více žádostí

jednoúrovňový systém



víceúrovňový systém



- **MCU STM8** využívají víceúrovňový systém se **3 úrovněmi** a
- možností nastavení **priorit** jednotlivých žádostí.

STM8 - Vektor přerušení

- je adresa (ukazatel) v paměti, na kterou procesor skočí při vzniku konkrétního přerušení, aby začal vykonávat obslužnou rutinu,
- naprosto základním **vektorem** je **RESET MCU** - určuje, odkud se začne vykonávat program viz obr.,

```

103 ;           vektory
104 ;-----
105
106         interrupt NonHandledInterrupt
107 NonHandledInterrupt.1
108         ired
109
110         segment 'vectit'
111         dc.l {$82000000+main}           ; reset
112         dc.l {$82000000+NonHandledInterrupt} ; trap
113         dc.l {$82000000+NonHandledInterrupt} ; irq0
114         dc.l {$82000000+NonHandledInterrupt} ; irq1
115         dc.l {$82000000+NonHandledInterrupt} ; irq2

```

- 0x82** je vyhrazený strojový kód vektoru následovaný 24 bit adresou **PCE:PCH:PCL**

STM8 - Zdroje přerušení

vektory přerušení				H	A	adresa	11		update/overflow/underflow	-	-	008034
-	RESET	reset	• •	008000	12	TIM1	compare/capture	-	-	008038		
-	TRAP	sw	- -	008004	13	TIM2	update/overflow	-	-	00803C		
0	TLI	externí top level	- -	008008	14		compare/capture	-	-	008040		
1	AWU	HALT	- -	00800C	15	TIM3	update/overflow	-	-	008044		
2	CLK	řízení hodin	- -	008010	16		compare/capture	-	-	008048		
3	EXTI0	externí PA *)	• •	008014	17	UART1	TX kompletní	-	-	00804C		
4	EXTI1	externí PB	• •	008018	18		Rec DATA FULL	-	-	008050		
5	EXTI2	externí PC	• •	00801C	19	I ² C	int	• •		008054		
6	EXTI3	externí PD	• •	008020	20		TX kompletní	-	-	008058		
7	EXTI4	externí PE	• •	008024	21	UART3	RX DATA FULL	-	-	00805C		
8	beCAN	RX	• •	008028	22	ADC2	end of conversion	-	-	008060		
9		TX/ER/SC	- -	00802C	23	TIM4	update/overflow	-	-	008064		
10	SPI	konec přenosu	• •	008030	24	Flash	EOP/WR_PG_DIS	-	-	008068		

- Adresa každého vektoru má přesně 4 byte (důvod viz předchozí slide),
- počátek vektorů je na adrese 0x8000,
- jeden vektor náleží jedné periférii, ale může reprezentovat více funkcí dané periférie (viz například **TIM1**, **TIM2**,...)

STM8 - Povolení přerušeni

- 1 Předně je nutné povolit přerušeni u dané periférie v příslušném registru (například u **ADC** je to **ADC_CSR**),
- 2 následně definovat prioritu, pokud využíváme víceúrovňový systém,
- 3 nakonec globálně povolit vybavení přerušeni (viz příznakový registr CC).

CC	V		I1	H	I0	N	Z	C
-----------	----------	--	-----------	----------	-----------	----------	----------	----------

Úrovně na základě stavu bitů **I1:I0**

I1	I0	úroveň	instrukce
1	0	hl. smyčka, přerušeni povoleno	RIM
0	1	úroveň 1	
0	0	úroveň 2, vyšší priorita	SIM
1	1	úroveň 3, přerušeni zakázáno	

STM8 - Nastavení priority přerušení

Registry ITC_SPRx:

	7	6	5	4	3	2	1	0
ITC_SPR1	VECT3SPR[1:0]		VECT2SPR[1:0]		VECT1SPR[1:0]		VECT0SPR[1:0]	
ITC_SPR2	VECT7SPR[1:0]		VECT6SPR[1:0]		VECT5SPR[1:0]		VECT4SPR[1:0]	
ITC_SPR3	VECT11SPR[1:0]		VECT10SPR[1:0]		VECT9SPR[1:0]		VECT8SPR[1:0]	
ITC_SPR4	VECT15SPR[1:0]		VECT14SPR[1:0]		VECT13SPR[1:0]		VECT12SPR[1:0]	
ITC_SPR5	VECT19SPR[1:0]		VECT18SPR[1:0]		VECT17SPR[1:0]		VECT16SPR[1:0]	
ITC_SPR6	VECT23SPR[1:0]		VECT22SPR[1:0]		VECT21SPR[1:0]		VECT20SPR[1:0]	
ITC_SPR7	VECT27SPR[1:0]		VECT26SPR[1:0]		VECT25SPR[1:0]		VECT24SPR[1:0]	
ITC_SPR8	Reserved				VECT29SPR[1:0]		VECT28SPR[1:0]	
	rw				rw	rw	rw	rw

- Každému vektoru odpovídají 2 bity - kombinace **I1:I0**,
- počáteční hodnota všech registrů po resetu je **0xFF**.

STM8 - Obsluha přerušení



- Kromě návratové adresy se do zásobníku ukládá také kontext programu, aby přerušení neměnilo hodnotu **CPU registrů** pro hlavní program,
- návrat z přerušení zajišťuje instrukce **IRET**

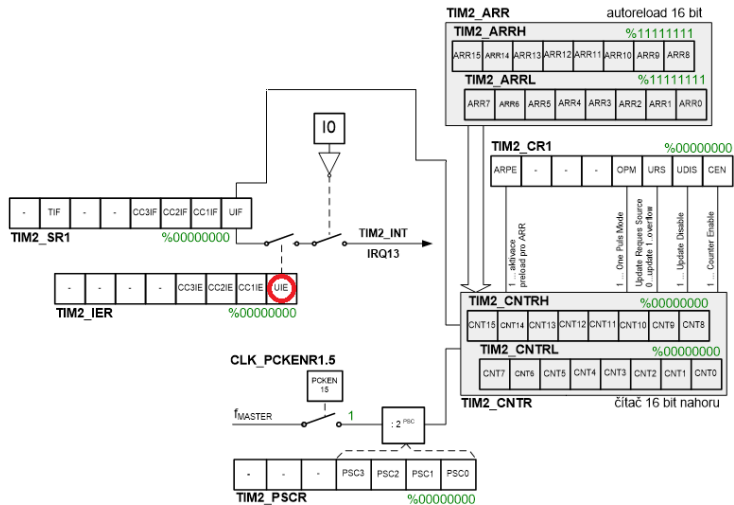
Téma

- 1 Funkce - podprogram
- 2 Rekurzivní funkce
- 3 Přerušení
- 4 Program TIM2 s přerušením**

Zadání

- 1 Použijeme program pro **TIM2** z předchozí přednášky,
- 2 upravíme hlavní smyčku tak, aby nevykonávala žádnou funkci,
- 3 blikání LED přesuneme do obsluhy přerušení přetečení **TIM2**,
- 4 přerušení povolíme a zaregistrujeme na příslušném vektoru.

Povolení přerušení periferie



Úprava definičních konstant

```

9  ;          DEFINICNI KONSTANTY
10 ;-----
11          |          BYTES
12 ; konstanty pro nastaveni pinu portu B = LED
13 INI_PB_DDR      equ      %11111111      ; vse vystup
14 INI_PB_CR1      equ      %11111111      ; vse push/pull
15 INI_PB_ODR      equ      %11111111      ; 1 zhasne LED
16
17 ; konstanty funkce wait
18 INI_TIM2PSCR     equ      6              ; preddelicka 512
19 INI_TIM2CR1      equ      %00000001     ; spustit citac
20 INI_TIM2IER      equ      %00000001     ; prerusení UIE
21
22          |          WORDS
23 INI_TIM2ARR      equ      15625         ; cilova hodnota

```

Přerušení UIE = 1

Úprava programu

```

38      ; inicializace TIM2
39      mov     TIM2_ARRH, #{high {INI_TIM2ARR}}
40      mov     TIM2_ARRL, #{low {INI_TIM2ARR}}
41      mov     TIM2_PSCR, #INI_TIM2PSCR
42      mov     TIM2_IER, #INI_TIM2IER
43      mov     TIM2_CR1, #INI_TIM2CR1
44
45      ; globalni povoleni preruseni
46      rim     Globální povolení přerušení
47
48
49      smycka  jp      smycka      Nic se nekoná      ; prazdna smycka
50
51      Obsluha
52      tim2isr bres     TIM2_SR1, #0      ; shozeni vlejky UIF
53      bcpl     PB_ODR, #0      ; negace bitu 0
54      ired
55

```

Zápis vektoru přerušení

```
63     segment 'vectit'
64     dc.l {$82000000+main}                ; reset
65     dc.l {$82000000+NonHandledInterrupt} ; trap
66     dc.l {$82000000+NonHandledInterrupt} ; irq0
67     dc.l {$82000000+NonHandledInterrupt} ; irq1
68     dc.l {$82000000+NonHandledInterrupt} ; irq2
69     dc.l {$82000000+NonHandledInterrupt} ; irq3
70     dc.l {$82000000+NonHandledInterrupt} ; irq4
71     dc.l {$82000000+NonHandledInterrupt} ; irq5
72     dc.l {$82000000+NonHandledInterrupt} ; irq6
73     dc.l {$82000000+NonHandledInterrupt} ; irq7
74     dc.l {$82000000+NonHandledInterrupt} ; irq8
75     dc.l {$82000000+NonHandledInterrupt} ; irq9
76     dc.l {$82000000+NonHandledInterrupt} ; irq10
77     dc.l {$82000000+NonHandledInterrupt} ; irq11
78     dc.l {$82000000+NonHandledInterrupt} ; irq12
79     dc.l {$82000000+tim2isr} Vektor TIM2 ; irq13
80     dc.l {$82000000+NonHandledInterrupt} ; irq14
81     dc.l {$82000000+NonHandledInterrupt} ; irq15
```

Následující přednáška

- SCADA,
- základy Control Web.

Děkuji za pozornost